



Backend Fundamentals

for AI Engineers

 **Build AI applications, not just AI models.**

! THE REALITY

Why Learn Backend?

Building an AI model is only one piece. Production AI needs APIs, authentication, databases, caching, streaming, background processing, monitoring, and scalable infrastructure.

YOU WILL LEARN

- ✓ Client & Server
- ✓ URLs & DNS
- ✓ HTTP & HTTPS
- ✓ REST APIs
- ✓ Authentication
- ✓ JWT & API Keys
- ✓ OAuth
- ✓ Cookies & Sessions
- ✓ CORS Security
- ✓ WebSockets
- ✓ Server-Sent Events
- ✓ Reverse Proxy
- ✓ Load Balancer
- ✓ Redis Caching
- ✓ Background Jobs
- ✓ Message Queues
- ✓ Async Programming
- ✓ Microservices
- ✓ Docker Deployment
- ✓ Logging & Metrics
- ✓ Rate Limiting
- ✓ Webhooks



Without backend fundamentals, it is difficult to build AI systems that are secure, scalable, and production-ready.

Client & Server

CLIENT

DEFINITION:

The application used by users to interact with your AI system.

EXAMPLES:

Browser, Mobile App, React Frontend.

AI EXAMPLE:

A user types a prompt into the ChatGPT website.

 **Why it Matters:** The client sends requests and displays AI responses.

SERVER

DEFINITION:

Receives requests, processes them, interacts with databases or AI models, and sends responses back.

AI EXAMPLE:

A FastAPI server receives the prompt and calls an LLM.

 **Key Takeaway:** The server does the heavy lifting behind every AI application.

URL Lifecycle & DNS

WHAT HAPPENS WHEN YOU TYPE A URL?

DEFINITION:

A URL starts a chain of events that connects your browser to the backend.

FLOW:

Browser → **DNS** → **Server** → **Backend** → **AI Model** → **Response**

AI EXAMPLE:

Typing `chat.openai.com` eventually reaches OpenAI's backend.

DNS (DOMAIN NAME SYSTEM)

DEFINITION:

DNS converts a human-readable domain into an IP address.

EXAMPLE:

`chat.openai.com` → `104.xxx.xxx.xxx`

 **Why it Matters:** Without DNS, browsers wouldn't know where your AI backend is located.

HTTP & HTTPS

HTTP

DEFINITION:

Transfers data between client and server.

AI EXAMPLE:

Sending a request to your FastAPI backend.

LIMITATION:

Data is not encrypted and can be intercepted.

HTTPS

DEFINITION:

Encrypts communication using SSL/TLS protocols.

AI EXAMPLE:

Protects user prompts, API keys, and uploaded documents.

 **Key Takeaway:** Every production AI application should use HTTPS.

HTTP Methods & Status Codes

HTTP METHODS

GET → Read

POST → Create

PUT → Replace

PATCH → Update

DELETE → Remove

AI EXAMPLE:

POST /chat

GET /history

HTTP STATUS CODES

200 Success

201 Created

400 Bad Request

401 Unauthorized

403 Forbidden

404 Not Found

429 Too Many Requests

500 Server Error

REST APIs & API Request Response

↔ REST APIS

DEFINITION:

A standard way for applications to communicate through defined endpoints.

🔗 AI EXAMPLE:

POST /chat

POST /embeddings

POST /images

📁 API REQUEST & RESPONSE

A REQUEST INCLUDES:

- URL
- Headers
- Query Parameters
- Body

A RESPONSE INCLUDES:

- Status Code
- Headers
- Response Body



CONCEPT 6

Authentication & Authorization



AUTHENTICATION

CORE QUESTION:

Who are you?

DEFINITION:

Verifies a user's identity before granting access.

EXAMPLES:

Password, Google Login, JWT



AUTHORIZATION

CORE QUESTION:

What can you access?

DEFINITION:

Controls permissions after authentication is successful.



AI EXAMPLE:

Admins can manage AI models, while standard users can only chat with them.

JWT Tokens & API Keys

JWT TOKEN

DEFINITION:

Keeps users logged in by securely storing identity information.

USED FOR:

User authentication.

 AI EXAMPLE:

A chatbot remembers the logged-in user across multiple chat requests.

API KEYS

DEFINITION:

Authenticate applications and services instead of human users.

USED FOR:

Accessing provider APIs securely.

 **Example: Authenticating with OpenAI, Anthropic, Gemini, or Groq APIs.**



OAuth & Cookies, Sessions, JWT

G OAUTH

DEFINITION:

Allows users to log in with trusted third-party providers.

EXAMPLE:

"Continue with Google" or "Continue with GitHub"

🔑 COOKIES, SESSIONS & JWT

Three common ways to manage user identity:

🔑 **COOKIES:** Stored directly in the browser.

📄 **SESSIONS:** Stored on the server for security.

🔑 **JWT:** Stored on the client and cryptographically verified by the server.

CORS & WebSockets

CORS

DEFINITION:

Controls which external websites are allowed to access your backend API.

AI EXAMPLE:

Configuring CORS to securely allow your React frontend to call your FastAPI backend.

WEBSOCKETS

DEFINITION:

Enable continuous, full-duplex, two-way communication between client and server.

BEST FOR:

- ✓ Live chat
- ✓ Real-time Voice AI
- ✓ Collaborative AI tools

Server-Sent Events & Comm. Patterns

"|" SERVER-SENT EVENTS (SSE)

DEFINITION:

Streams data continuously from the server to the client in real time.

🧩 AI EXAMPLE:

ChatGPT streaming its response back to you one token at a time!

🔗 WHICH PATTERN TO CHOOSE?

Pattern	Best For
Polling	Periodic updates
Long Polling	Delayed updates
WebSockets	Two-way real time
SSE	One-way streaming

Reverse Proxy & Load Balancer

REVERSE PROXY

DEFINITION:

Receives incoming requests first and forwards them securely to backend servers.

POPULAR TOOL:

Nginx

LOAD BALANCER

DEFINITION:

Distributes massive traffic evenly across multiple backend servers.

★ **Benefit:** Ensures scalability and high availability, keeping your AI app from crashing during spikes.

Caching & Background Jobs

CACHING

DEFINITION:

Stores frequently used data in memory for lightning-fast access.

POPULAR TOOL:

Redis

AI EXAMPLE:

Caching embeddings or previous identical responses to save API costs.

BACKGROUND JOBS

DEFINITION:

Run time-consuming tasks asynchronously without blocking the user interface.

AI EXAMPLE:

Generating embeddings for a 100-page PDF document immediately after upload.

Message Queues & Async Programming

MESSAGE QUEUES

DEFINITION:

Process and distribute heavy tasks reliably in the background across workers.

EXAMPLES:

✓ Kafka ✓ RabbitMQ ✓ Amazon SQS

ASYNC PROGRAMMING

DEFINITION:

Allows the server to handle many concurrent requests simultaneously without blocking.

AI EXAMPLE:

Serving multiple chat requests while waiting for slow LLM responses to arrive.



Microservices & Docker Deployment



MICROSERVICES

DEFINITION:

Split large monolithic applications into smaller, manageable, independent services.



AI SYSTEM ARCHITECTURE:

Authentication Service | Inference Engine | Billing Module | Notifications



DOCKER & DEPLOYMENT PIPELINE

DEFINITION:

Package applications into containers to run identically everywhere.

DEPLOYMENT FLOW:

Local → Docker → Cloud → Production

Logging, Monitoring & Webhooks

LOGGING & MONITORING

DEFINITION:

Track application health and diagnose production issues rapidly.

MONITOR THESE METRICS:

- ✓ Latency
- ✓ Errors
- ✓ Token Usage
- ✓ CPU / Memory

RATE LIMITING & WEBHOOKS

RATE LIMITING:

Protects APIs from abuse and cost overruns by controlling request frequency.

WEBHOOKS:

Receive events automatically instead of continuously checking an API.

AI EXAMPLE:

Get notified instantly via webhook when a fine-tuning job finishes!